

Báo cáo Bài tập về nhà ngày 26/05/2026

Họ và tên: Nguyễn Thành Nam

MSSV: 20225368

Bài 1. Tạo trang web thực hiện các phép tính cộng, trừ, nhân, chia và trả lại kết quả cho trình duyệt:

+ Nhận tham số từ lệnh GET

+ Nhận tham số từ lệnh POST

+ Các tham số bao gồm toán tử (cộng, trừ, nhân, chia) và 2 toán hạng

+ Server trả lại kết quả là trang web hiển thị phép tính và kết quả.

Mã nguồn:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h>
```

```
#define PORT 5000
```

```
#define BUFFER_SIZE 4096
```

```
void get_param(const char *src, const char *key, char *dest) {
```

```
    char *start = strstr(src, key);
```

```
    if (start) {
```

```
        start += strlen(key);
```

```

char *end = strchr(start, '&');
if (!end) end = strchr(start, ' ');
if (end) {
    strncpy(dest, start, end - start);
    dest[end - start] = '\0';
} else {
    strcpy(dest, start);
}
} else {
    dest[0] = '\0';
}
}

```

```

void do_calculation(char *a_str, char *b_str, char *op_str, char *result_str) {
    if (strlen(a_str) == 0 || strlen(b_str) == 0 || strlen(op_str) == 0) {
        strcpy(result_str, "Chưa nhận dữ liệu phép tính hợp lệ.");
        return;
    }
    double a = atof(a_str);
    double b = atof(b_str);

    if (strcmp(op_str, "cong") == 0 || strcmp(op_str, "%2B") == 0)
        sprintf(result_str, "Kết quả: %.2f + %.2f = %.2f", a, b, a + b);
    else if (strcmp(op_str, "tru") == 0)
        sprintf(result_str, "Kết quả: %.2f - %.2f = %.2f", a, b, a - b);
    else if (strcmp(op_str, "nhan") == 0)

```

```

        sprintf(result_str, "Kết quả: %.2f * %.2f = %.2f", a, b, a * b);
else if (strcmp(op_str, "chia") == 0) {
    if (b != 0) sprintf(result_str, "Kết quả: %.2f / %.2f = %.2f", a, b, a / b);
    else strcpy(result_str, "Lỗi: Không thể thực hiện phép chia cho giá trị 0!");
} else {
    sprintf(result_str, "Toán tử '%s' không được hỗ trợ hệ thống.", op_str);
}
}

int main() {
    int server_fd, client_fd;

    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 10);
    printf("[SERVER] Máy tính HTTP phục vụ hoạt động tại cổng: %d...\n", PORT);

```

```

while (1) {

    client_fd = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);

    memset(buffer, 0, BUFFER_SIZE);

    int bytes_recv = read(client_fd, buffer, BUFFER_SIZE - 1);

    if(bytes_recv < 0) { close(client_fd); continue; }


    char a_str[32] = "", b_str[32] = "", op_str[32] = "", result[256] = "";


    if (strncmp(buffer, "GET /calc", 9) == 0) {

        get_param(buffer, "a=", a_str);

        get_param(buffer, "b=", b_str);

        get_param(buffer, "op=", op_str);

        do_calculation(a_str, b_str, op_str, result);

    }

    else if (strncmp(buffer, "POST /calc", 10) == 0) {

        char *body = strstr(buffer, "\r\n\r\n");

        if (body) {

            body += 4;

            get_param(body, "a=", a_str);

            get_param(body, "b=", b_str);

            get_param(body, "op=", op_str);

            do_calculation(a_str, b_str, op_str, result);

        }

    }

}

```

```

char html[BUFFER_SIZE];

char body_content[1024];

sprintf(body_content,
    "<h2>MÁY TÍNH HTTP - NGÔN NGỮ C</h2>"
    "<form method='POST' action='/calc'>"
    " <input type='number' step='any' name='a' placeholder='Số thứ nhất' required> "
    " <select name='op'>"
    " <option value='cong'>+ (Cộng)</option>"
    " <option value='tru'>- (Trừ)</option>"
    " <option value='nhan'>* (Nhân)</option>"
    " <option value='chia'>/ (Chia)</option>"
    " </select> "
    " <input type='number' step='any' name='b' placeholder='Số thứ hai' required> "
    " <button type='submit'>Tính toán (POST)</button>"
    "</form>"
    "<div style='margin-top:20px; padding:10px; background:#e8f4fd; border-radius:4px;'>"
    " <strong>%s</strong>"
    "</div>"
    "<p><small>Thử nghiệm lệnh GET trực tiếp qua URL: <a href='/calc?a=12.5&b=4&op=nhan'>/calc?a=12.5&b=4&op=nhan</a></small></p>",
    result);

sprintf(html,
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html; charset=UTF-8\r\n"

```

```

        "Content-Length: %d\r\n"

        "Connection: close\r\n\r\n"

        "<!DOCTYPE html><html><head><title>HTTP Calculator in
C</title><style>body{font-
family:Arial;margin:40px;}</style></head><body>%s</body></html>",

        (int)(strlen(body_content) + 160), body_content);

    send(client_fd, html, strlen(html), 0);

    close(client_fd);
}

return 0;
}

```

Giải thích mã nguồn:

Mã nguồn Bài 1 thực hiện nhiệm vụ khởi tạo một máy chủ web ở mức lập trình hệ thống (low-level), tự động bóc tách các tham số số học từ giao thức HTTP (phương thức GET và POST) để xử lý tính toán và trả về giao diện trực quan cho người dùng.

1. Cấu trúc các thư viện sử dụng:

- #include <stdio.h> & #include <stdlib.h>: Cung cấp các hàm nhập xuất cơ bản (sprintf, printf) và hàm chuyển đổi kiểu dữ liệu chuỗi thành số thực (atof).
- #include <string.h>: Cung cấp các hàm thao tác và xử lý chuỗi thô (strstr, strchr, strcmp, strncmp, strlen).
- #include <unistd.h>: Chứa hàm read() và close() để tương tác với các tệp tin mô tả (descriptor) của Socket hệ thống.
- #include <sys/socket.h> & #include <netinet/in.h>: Bộ thư viện cốt lõi định nghĩa cấu trúc dữ liệu địa chỉ Internet (sockaddr_in) và các hàm thiết lập mạng Socket TCP.

2. Hàm trích xuất tham số dữ liệu (get_param):

- Chức năng: Hàm này chịu trách nhiệm tìm kiếm một từ khóa (tham số) như "a=", "b=", hoặc "op=" trong chuỗi dữ liệu HTTP Request và trích xuất giá trị đi kèm của từ khóa đó.

- Nguyên lý hoạt động:

+ Hàm sử dụng strstr(src, key) để định vị vị trí bắt đầu của từ khóa trong chuỗi gốc.

+ Nếu tìm thấy, con trỏ dịch chuyển qua độ dài của từ khóa để đến vị trí của giá trị cần lấy.

+ Tiếp theo, hàm sử dụng strchr(start, '&') hoặc strchr(start, ' ') để tìm ký tự kết thúc của tham số đó (các tham số trong HTTP thường phân cách nhau bằng dấu "&" hoặc kết thúc bằng khoảng trắng khi nằm ở cuối dòng trạng thái).

+ Cuối cùng, hàm sao chép phần giá trị đã cắt vào biến đích dest và thêm ký tự kết thúc chuỗi '\0'.

3. Hàm logic tính toán số học (do_calculation):

- Chức năng: Tiếp nhận các chuỗi tham số thô thu được từ Request, chuyển đổi sang số thực, kiểm tra toán tử và thực hiện phép tính số học tương ứng.

- Nguyên lý hoạt động:

+ Đầu tiên, hàm kiểm tra nếu một trong ba tham số bị bỏ trống thì sẽ báo lỗi chưa nhận đủ dữ liệu.

+ Hàm sử dụng atof(a_str) và atof(b_str) để ép kiểu chuỗi ký tự sang số thực double, giúp máy tính có thể xử lý được cả số nguyên lẫn số thập phân một cách chính xác.

+ Hàm sử dụng strcmp() để so sánh chuỗi toán tử op_str:

* Nếu là "cong" (hoặc mã hóa URL "%2B"), thực hiện phép cộng (+).

* Nếu là "tru", thực hiện phép trừ (-).

* Nếu là "nhan", thực hiện phép nhân (*).

* Nếu là "chia", hàm tiến hành kiểm tra điều kiện mẫu số $b \neq 0$. Nếu $b == 0$, hàm lập tức ghi nhận cảnh báo lỗi chia cho 0 để tránh việc CPU xử lý lỗi gây sập chương trình. Ngược lại, thực hiện phép chia (/).

+ Toàn bộ kết quả định dạng số thực rút gọn 2 chữ số thập phân (%.2f) được ghi vào chuỗi result_str qua hàm sprintf().

4. Hàm khởi tạo và luồng xử lý chính (main):

- Cấu hình Socket: Máy chủ sử dụng `socket(AF_INET, SOCK_STREAM, 0)` để tạo socket TCP. Lệnh `setsockopt()` với tham số `SO_REUSEADDR` giúp Server có thể giải phóng và tái sử dụng cổng Port ngay lập tức khi khởi động lại. Hàm `bind()` thực hiện gán socket với cổng 5000 và `listen()` thiết lập hàng đợi kết nối.

- Xử lý HTTP Request trong vòng lặp `while(1)`:

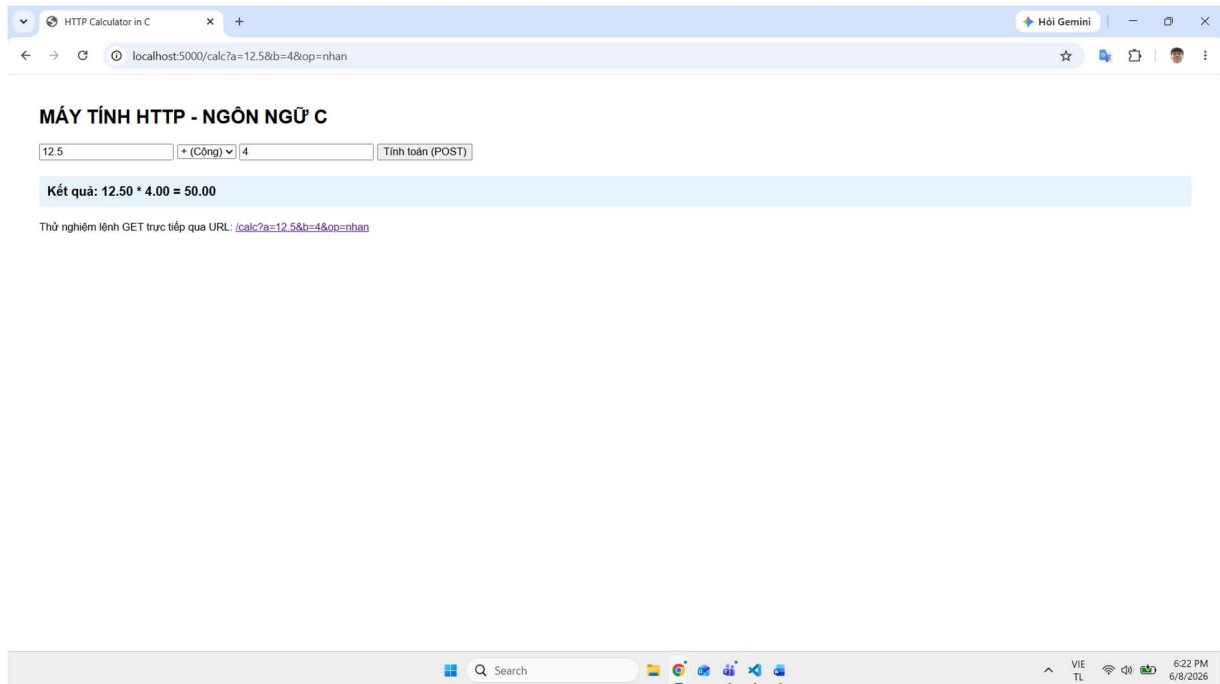
+ Khi nhận được kết nối qua `accept()`, lệnh `read()` sẽ đọc gói tin Request thô lưu vào buffer.

+ Phân tích GET: Nếu chuỗi bắt đầu bằng "GET /calc", Server gọi trực tiếp hàm `get_param` trên chuỗi buffer vì tham số lệnh GET được truyền tường minh ngay dòng đầu tiên của Header trên URL.

+ Phân tích POST: Nếu chuỗi bắt đầu bằng "POST /calc", tham số sẽ ẩn trong HTTP Body. Server dùng `strstr(buffer, "\r\n\r\n")` để tìm dòng trống phân tách, dịch con trỏ xuống 4 ký tự (nhảy qua dữ liệu "\r\n\r\n") để tiếp cận phần Body và trích xuất tham số.

- Phản hồi dữ liệu (HTTP Response): Sau khi tính toán, mã HTML giao diện form và chuỗi kết quả được gom lại. Máy chủ đóng gói một HTTP Response chuẩn bao gồm Header chứa dòng trạng thái "HTTP/1.1 200 OK", định dạng "Content-Type: text/html", độ dài "Content-Length" và bắn luồng dữ liệu này về trình duyệt bằng hàm `send()`. Cuối cùng kết nối được đóng lại bằng `close(client_fd)` để giải phóng tài nguyên.

Kết quả:



Bài 2. Lập trình ứng dụng máy chủ HTTP file với các chức năng:

- + Khi trình duyệt yêu cầu thư mục gốc, trả về nội dung là trang web chứa danh sách các liên kết đến thư mục và tập tin của thư mục hiện tại trên server. In đậm đối với thư mục và in nghiêng đối với file.
- + Nếu người dùng nhấn vào liên kết thư mục, server trả về nội dung của thư mục con đó (tương tự như trên)
- + Nếu người dùng nhấn vào liên kết file, thì trả về nội dung file. Các file được hỗ trợ bao gồm file văn bản, file audio, file video, file ảnh.

Mã nguồn:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <stddef.h>

#include <unistd.h>
```

```

#include <dirent.h>

#include <sys/stat.h>

#include <sys/socket.h>

#include <netinet/in.h>


#define PORT 8080

#define BUFFER_SIZE 8192

#define ROOT_DIR "./shared_folder"


const char* get_mime_type(const char *filename) {
    if (strstr(filename, ".txt") || strstr(filename, ".c") || strstr(filename, ".h"))
        return "text/plain; charset=UTF-8";
    if (strstr(filename, ".png")) return "image/png";
    if (strstr(filename, ".jpg") || strstr(filename, ".jpeg")) return "image/jpeg";
    if (strstr(filename, ".mp3")) return "audio/mpeg";
    if (strstr(filename, ".mp4")) return "video/mp4";
    return "application/octet-stream";
}


void send_directory_listing(int client_fd, const char *current_subpath) {
    char target_dir[2048];
    if (strlen(current_subpath) == 0) {
        sprintf(target_dir, "%s", ROOT_DIR);
    } else {
        sprintf(target_dir, "%s/%s", ROOT_DIR, current_subpath);
    }
}

```

```

DIR *d = opendir(target_dir);

char response_body[65536] = "";

sprintf(response_body,
    "<html><head><meta charset='UTF-8'><title>HTTP File Server in C</title>"
    "<style>body{font-family:Arial, sans-serif; margin:40px;} li{margin:10px 0; font-size:13pt;} a{text-decoration:none; color:#0066cc;}</style></head>"
    "<body><h2>Danh sách tệp tin và thư mục: /%s</h2><ul>", current_subpath);

if (strlen(current_subpath) > 0) {
    strcat(response_body, "<li><a href='/'><b>[..] Quay về Thư mục gốc</b></a></li>");
}

if (d) {
    struct dirent *dir;
    while ((dir = readdir(d)) != NULL) {
        if (strcmp(dir->d_name, ".") == 0 || strcmp(dir->d_name, "..") == 0) continue;

        char full_item_path[4096];
        sprintf(full_item_path, "%s/%s", target_dir, dir->d_name);

        struct stat st;
        stat(full_item_path, &st);

        char item_link[8192];

```

```

char web_path[2048];

if (strlen(current_subpath) == 0) {
    sprintf(web_path, "%s", dir->d_name);
} else {
    sprintf(web_path, "%s/%s", current_subpath, dir->d_name);
}

if (S_ISDIR(st.st_mode)) {
    sprintf(item_link, "<li><a href=\"\"/files/%s\"><b>%s</b></a></li>", web_path, dir-
>d_name);
} else if (S_ISREG(st.st_mode)) {
    sprintf(item_link, "<li><a href=\"\"/files/%s\"><i>%s</i></a></li>", web_path, dir-
>d_name);
} else {
    continue;
}

strcat(response_body, item_link);
}

closedir(d);

} else {
    strcat(response_body, "<p style='color:red;'>Lỗi: Không thể truy cập hoặc thư mục
không tồn tại.</p>");
}

strcat(response_body, "</ul></body></html>");

char response_header[1024];

```

```

sprintf(response_header,
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: %d\r\n"
    "Connection: close\r\n\r\n", (int)strlen(response_body));

send(client_fd, response_header, strlen(response_header), 0);
send(client_fd, response_body, strlen(response_body), 0);
}

void send_file_content(int client_fd, const char *file_relative_path) {
    char full_file_path[4096];
    sprintf(full_file_path, "%s/%s", ROOT_DIR, file_relative_path);

    FILE *f = fopen(full_file_path, "rb");
    if (!f) {
        char *not_found = "HTTP/1.1 404 Not Found\r\nContent-Length: 29\r\nConnection:
close\r\n\r\nTập tin không tồn tại trên hệ thống.";
        send(client_fd, not_found, strlen(not_found), 0);
        return;
    }

    struct stat st;
    stat(full_file_path, &st);
    long file_size = st.st_size;

```

```

char header[1024];

sprintf(header,

    "HTTP/1.1 200 OK\r\n"

    "Content-Type: %s\r\n"

    "Content-Length: %ld\r\n"

    "Connection: close\r\n\r\n", get_mime_type(file_relative_path), file_size);

send(client_fd, header, strlen(header), 0);


char file_buffer[2048];

int bytes_read;

while ((bytes_read = fread(file_buffer, 1, sizeof(file_buffer), f)) > 0) {

    send(client_fd, file_buffer, bytes_read, 0);

}

fclose(f);
}


int main() {

    int server_fd, client_fd;

    struct sockaddr_in address;

    int opt = 1;

    int addrlen = sizeof(address);

    char buffer[BUFFER_SIZE] = {0};


#ifdef _WIN32

    mkdir(ROOT_DIR);

#else

```

```
mkdir(ROOT_DIR, 0777);

#endif

server_fd = socket(AF_INET, SOCK_STREAM, 0);

if (server_fd < 0) {
    perror("Khởi tạo Socket thất bại");
    exit(EXIT_FAILURE);
}

setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
    perror("Lỗi gán cổng Bind thất bại");
    close(server_fd);
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 10) < 0) {
    perror("Lỗi thiết lập Listen");
    close(server_fd);
    exit(EXIT_FAILURE);
}
```

```
printf("[SERVER] Máy chủ File Server HTTP hoạt động tại cổng: %d...\n", PORT);
```

```
while (1) {
```

```
    client_fd = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);
```

```
    if (client_fd < 0) continue;
```

```
    memset(buffer, 0, BUFFER_SIZE);
```

```
    int bytes_recv = read(client_fd, buffer, BUFFER_SIZE - 1);
```

```
    if (bytes_recv <= 0) {
```

```
        close(client_fd);
```

```
        continue;
```

```
    }
```

```
    char method[16] = "", url[1024] = "", version[16] = "";
```

```
    sscanf(buffer, "%s %s %s", method, url, version);
```

```
    if (strcmp(url, "/") == 0 || strcmp(url, "/files") == 0 || strcmp(url, "/files/") == 0) {
```

```
        send_directory_listing(client_fd, "");
```

```
    }
```

```
    else if (strncmp(url, "/files/", 7) == 0) {
```

```
        char *subpath = url + 7;
```

```
        char full_check_path[4096];
```

```
        sprintf(full_check_path, "%s/%s", ROOT_DIR, subpath);
```

```
        struct stat st;
```



```

if (stat(full_check_path, &st) == 0) {
    if (S_ISDIR(st.st_mode)) {
        send_directory_listing(client_fd, subpath);
    } else if (S_ISREG(st.st_mode)) {
        send_file_content(client_fd, subpath);
    }
} else {
    char *not_found = "HTTP/1.1 404 Not Found\r\nContent-Length: 20\r\n\r\nĐường
dẫn không đúng";
    send(client_fd, not_found, strlen(not_found), 0);
}
} else {
    char *bad_req = "HTTP/1.1 400 Bad Request\r\nContent-Length: 15\r\n\r\nYêu cầu
sai lỗi";
    send(client_fd, bad_req, strlen(bad_req), 0);
}

close(client_fd);
}

close(server_fd);
return 0;
}

```

Giải thích mã nguồn:

1. Các thư viện hệ thống chuyên dụng:

- dirent.h: Cung cấp kiểu dữ liệu cấu trúc DIR và cấu trúc dòng thực thể dirent, cùng bộ đôi hàm mở thư mục opendir() và hàm đọc tuần tự readdir().

- sys/stat.h: Chứa hàm stat() và cấu trúc dữ liệu st dùng để truy vấn các thuộc tính vật lý mức thấp của tệp tin hoặc thư mục như kích thước, quyền truy cập và phân loại thực thể.

2. Hàm nhận diện định dạng tệp tin (get_mime_type):

- Chức năng: Hàm tiếp nhận tham số là tên tệp tin khách hàng yêu cầu, thực hiện quét phần mở rộng (đuôi file) bằng hàm strstr() để trả về chuỗi mô tả Content-Type chuẩn quốc tế.

- Vai trò: Việc định nghĩa chính xác MIME-Type (ví dụ: image/png cho ảnh, audio/mpeg cho nhạc, video/mp4 cho video) đóng vai trò quyết định giúp Trình duyệt Web kích hoạt ngay các trình phát đa phương tiện tích hợp sẵn, thay vì tự động tải tệp tin về máy dưới dạng nhị phân thô không xác định.

3. Hàm sinh giao diện cây liên kết (send_directory_listing):

- Chức năng: Hàm mở thư mục mục tiêu trên máy chủ bằng lệnh opendir(). Nếu mở thành công, vòng lặp while kết hợp readdir() sẽ duyệt qua từng mục dữ liệu bên trong.

- Thực hiện yêu cầu đề bài: Đối với mỗi thực thể tìm thấy, hàm stat() được gọi để bóc tách thuộc tính mặt nạ bit hệ thống:

+ Nếu macro S_ISDIR(st.st_mode) trả về TRUE (đây là thư mục): Tên thực thể được bọc trong thẻ HTML (In đậm theo yêu cầu) và gắn kèm ký tự gạch chéo phân định (/).

+ Nếu macro S_ISREG(st.st_mode) trả về TRUE (đây là tệp tin thường): Tên thực thể được bọc trong thẻ HTML <i> (In nghiêng theo yêu cầu).

- Cơ chế gom chuỗi: Toàn bộ các thẻ liên kết này được nối liên tục vào mảng bộ nhớ đệm response_body có kích thước lớn (65536 bytes) thông qua hàm strcat(), tạo thành giao diện một trang Web danh sách. Cuối cùng, Server gửi gói Header HTTP 200 OK và Body chứa trang HTML này về cho trình duyệt.

4. Hàm đọc ghi nhị phân tối ưu hóa (send_file_content):

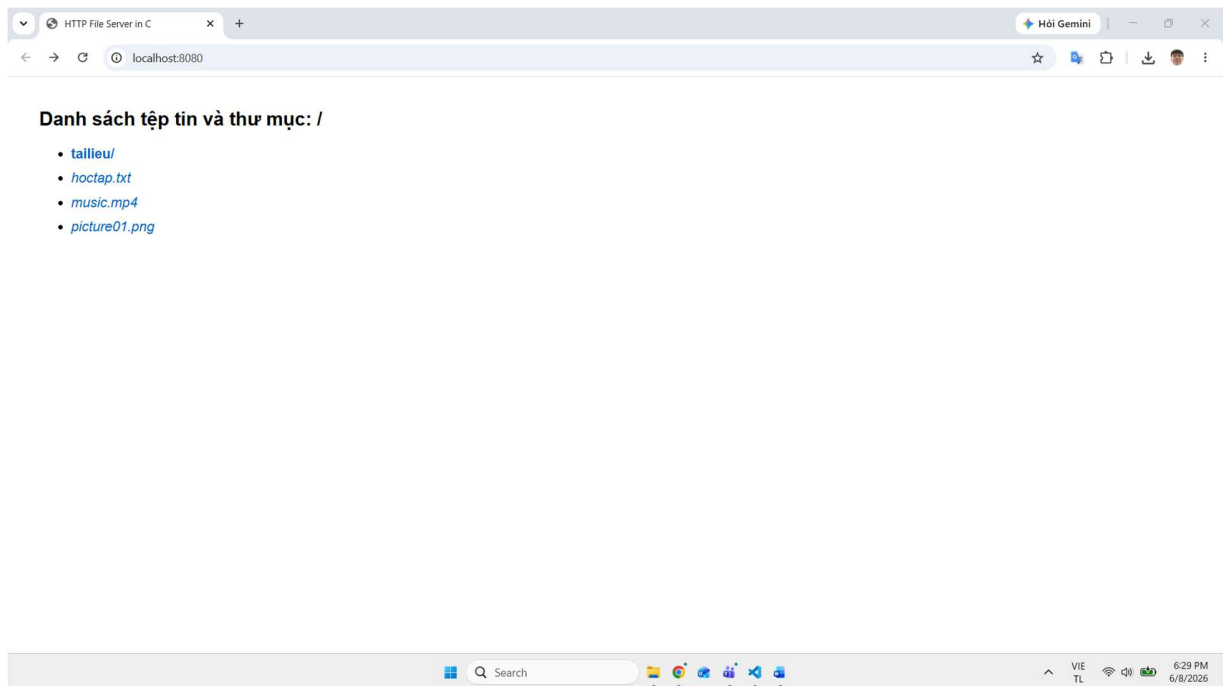
- Chức năng: Khi người dùng click vào một liên kết là tệp tin, hàm này chịu trách nhiệm nạp dữ liệu từ ổ cứng và truyền luồng (Stream) về Socket mạng.

- Cơ chế đọc an toàn: Hàm bắt buộc sử dụng chế độ mở tệp tin nhị phân nguyên bản "rb" (Read Binary) của lệnh fopen() để đảm bảo cấu trúc dữ liệu của các file ảnh, nhạc, video không bị lỗi hoặc sai lệch ký tự xuống dòng như chế độ đọc văn bản thông thường.
- Cơ chế truyền tải dữ liệu: Hàm sử dụng cấu trúc stat để đo đạc chính xác kích thước file (st.st_size), ghi nhận giá trị này vào Header Content-Length để Trình duyệt biết trước dung lượng. Sau đó, một vòng lặp tuần tự sử dụng fread() sẽ liên tục bốc từng khối dữ liệu kích thước 2048 bytes đặt vào mảng file_buffer và bắn ngay sang hàm send() cho đến khi tệp tin được truyền tải trọn vẹn, giúp tối ưu hóa dung lượng RAM của Server.

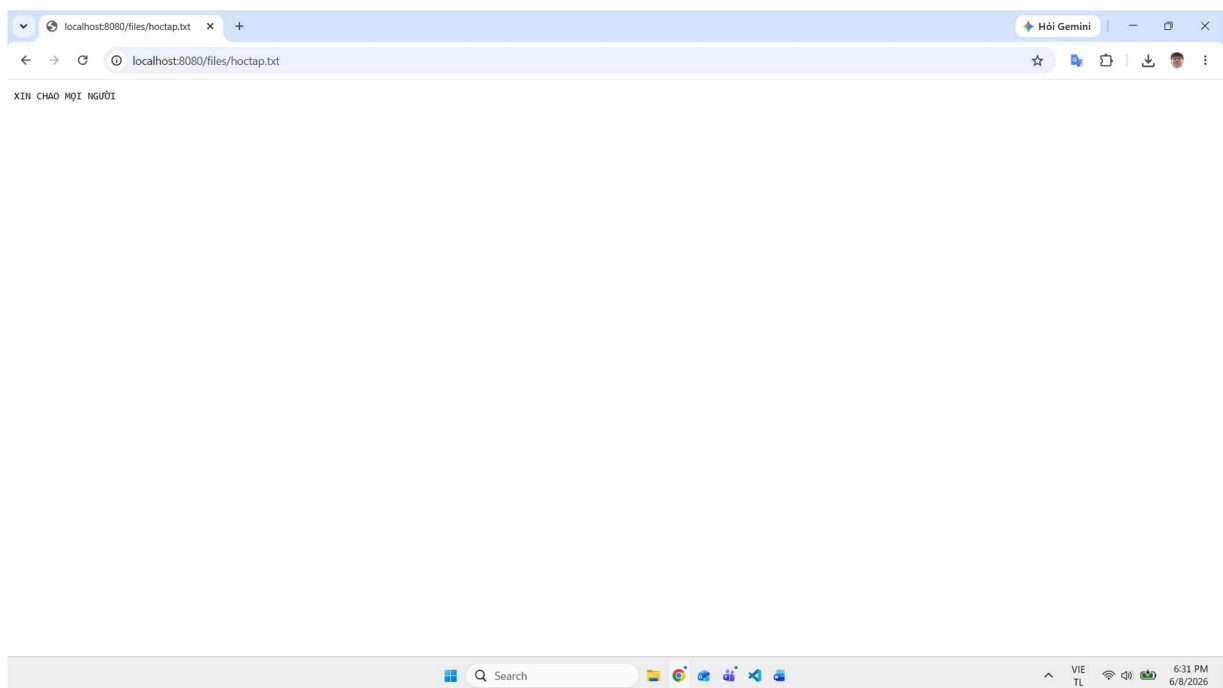
5. Luồng hoạt động tại hàm chính (main):

- Server khởi tạo cấu trúc Socket mạng TCP trên cổng Port 8080 và gọi lệnh lắng nghe listen().
- Khi trình duyệt gửi Request lên, lệnh sscanf(buffer, "%s %s %s", method, url, version) sẽ bóc tách chuỗi để lấy chính xác địa chỉ URI (biến url).
- Nếu url là "/" hoặc "/files/", máy chủ hiểu người dùng muốn duyệt thư mục gốc và gọi hàm send_directory_listing().
- Nếu url bắt đầu bằng tiền tố "/files/[đường_dẫn_con]", máy chủ sẽ bỏ qua 7 ký tự tiền tố để lấy ra đường dẫn thực tế, dùng hàm stat() kiểm tra xem thực thể đó là thư mục con hay tập tin cụ thể để điều hướng gọi hàm xử lý tương ứng (trả về danh sách thư mục con hoặc trả về nội dung file phương tiện).
- Sau khi hàm send() hoàn tất việc đẩy dữ liệu, lệnh close(client_fd) lập tức được kích hoạt để giải phóng tài nguyên hệ thống.

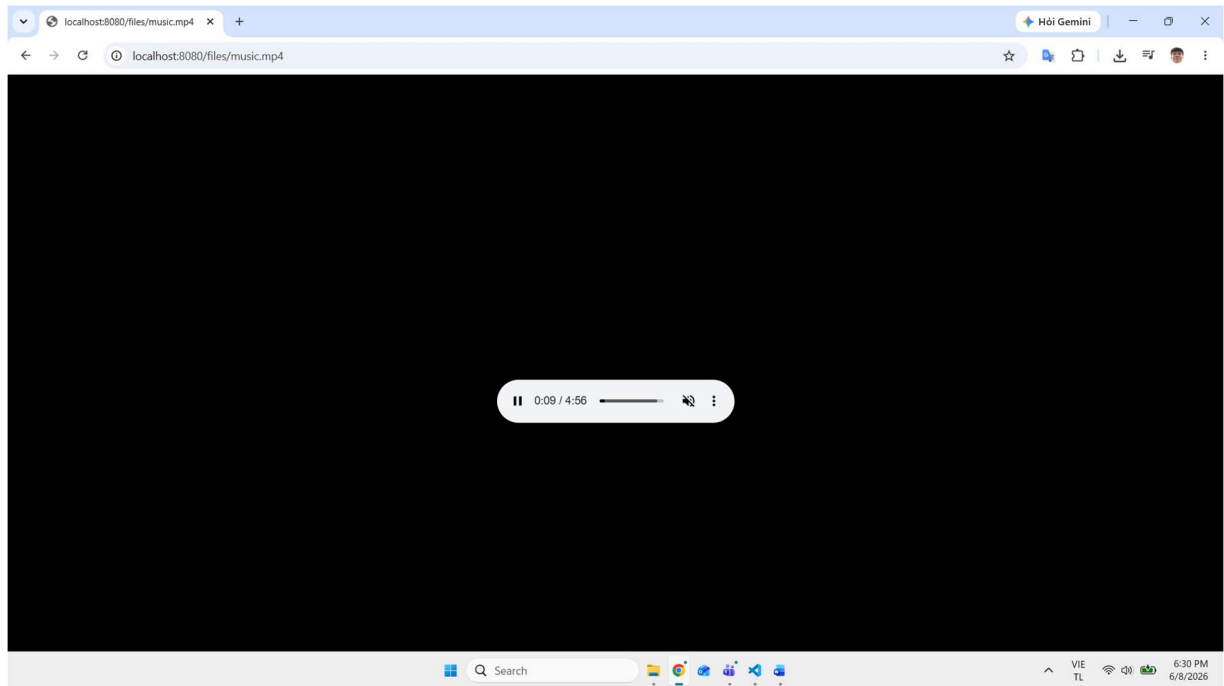
Kết quả:



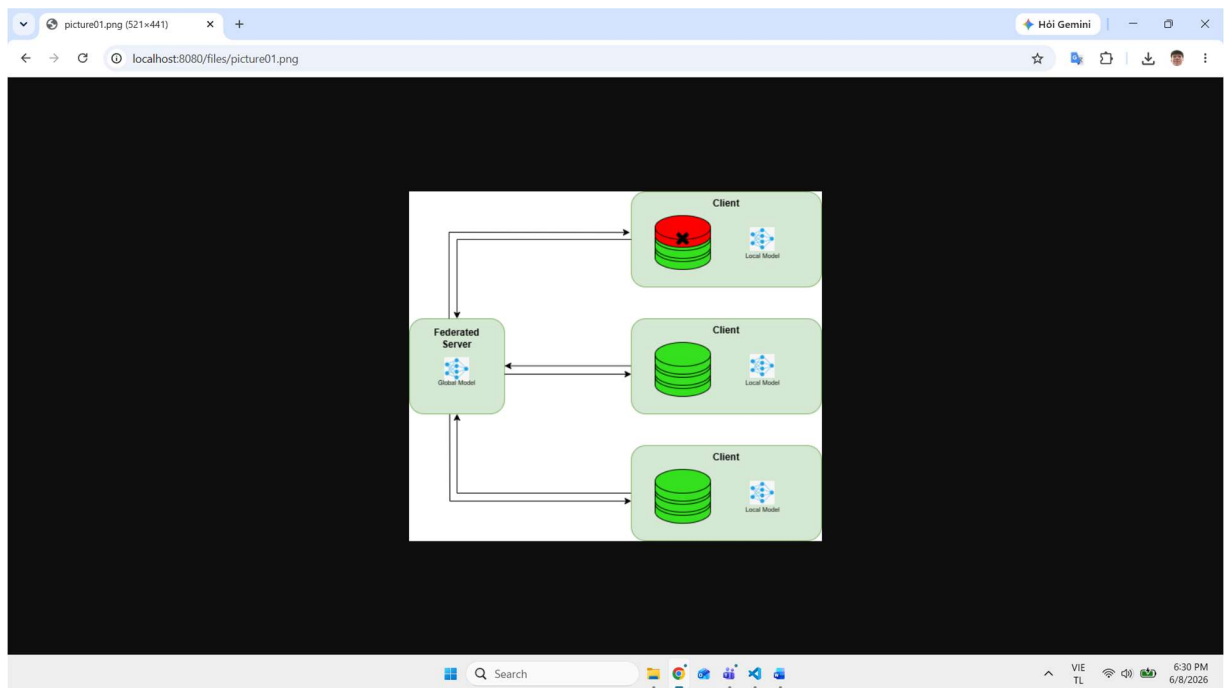
Hình 1. Giao diện Terminal hệ điều hành hiển thị trạng thái máy chủ File Server hoạt động thành công tại cổng 8080.



Hình 2. Hiện thị file văn bản



Hình 3. Hiện thị file mp4



Hình 4. Hiện thị file ảnh